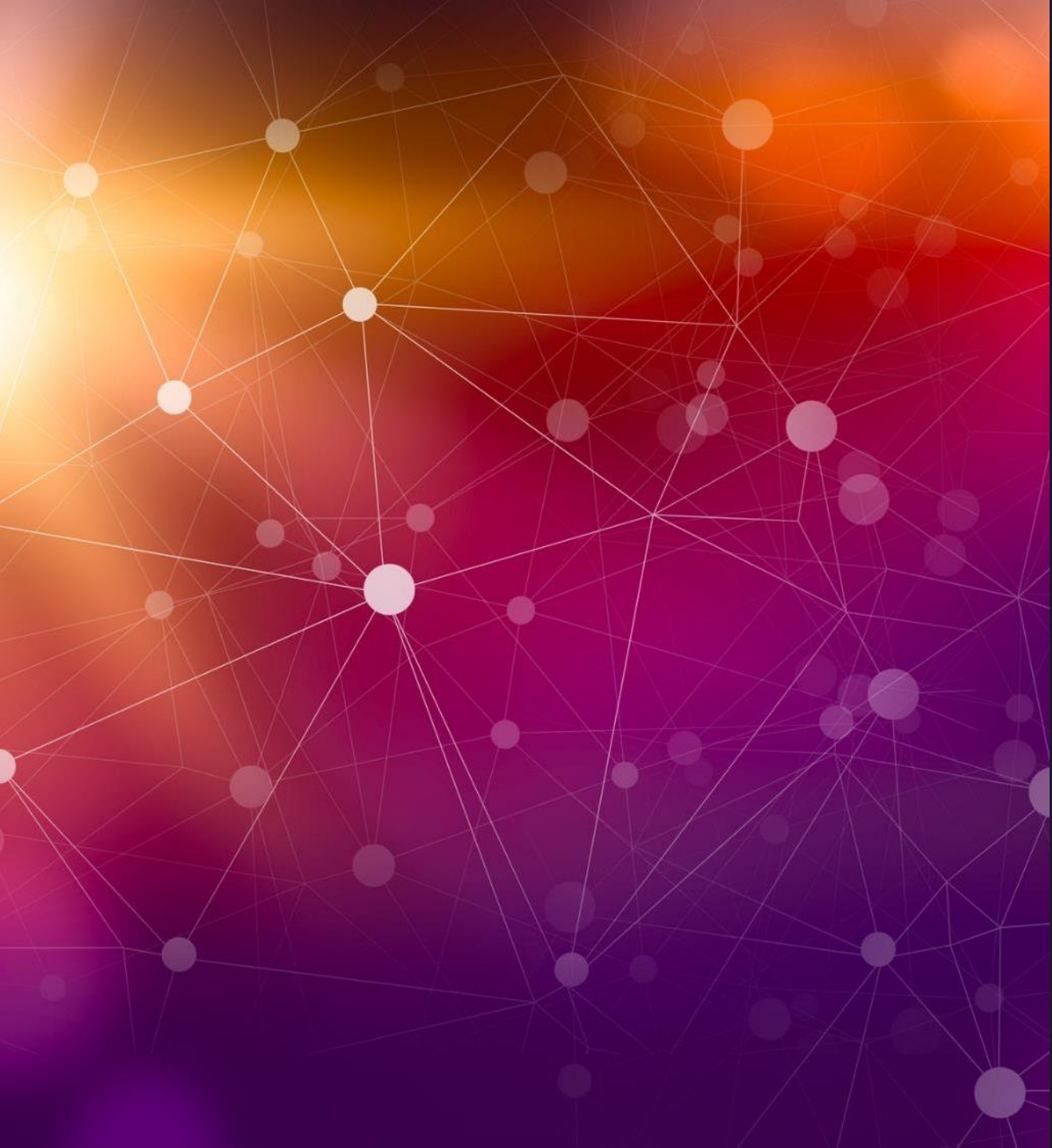




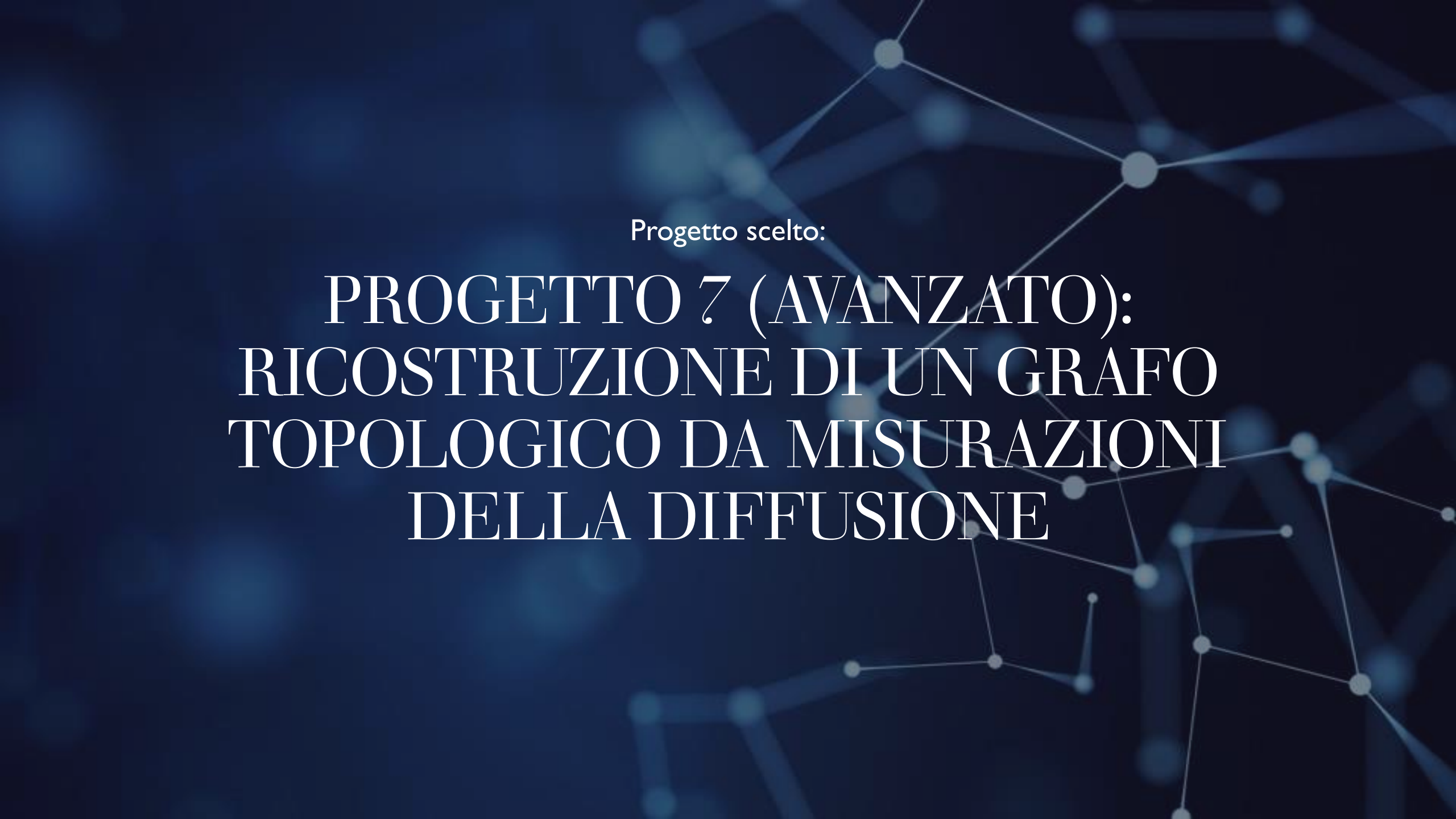
Presentazione progetto Calcolo Numerico

Gruppo: Paduret Alessandro, Singh Alex, Stupu David, Valente Ivan

A.A. 2025-2026

Corso di Laurea Triennale Informatica
Dipartimento di Matematica, Scuola di Scienze

4 febbraio 2026



Progetto scelto:

PROGETTO 7 (AVANZATO): RICOSTRUZIONE DI UN GRAFO TOPOLOGICO DA MISURAZIONI DELLA DIFFUSIONE

Sommario

- Contesto teorico e scopo del progetto
- Inizializzazione
- Problema 1: dati diffusivi sovrabbondanti
- Problema 2: dati diffusivi lacunosi
- Considerazioni finali





Contesto teorico e scopo del progetto

Contesto: Laplaciano

- Operatore di Laplace
- Misura quanto il valore di una funzione in un punto differisce dalla media locale
- Ottima applicazione pratica: modellazione dei processi diffusivi

$$\Delta f(x) := \sum_{i=1}^n \frac{\partial^2}{\partial x_i^2} f(x)$$

$$f(\bar{x}) = \frac{1}{|B(\bar{x}, r)|} \int_{B(\bar{x}, r)} f(x) dx$$



Contesto: Equazione del calore

- Perfetto esempio di processo diffusivo
- Dispersione omnidirezionale
- Effetto immediato anche a distanza infinita
- Decadimento rapido

- $$\begin{cases} \frac{\partial}{\partial t} u(x, t) = \Delta u(x, t) & x \in \mathbb{R}, t > 0 \\ u(x, 0) = u^0(x) & x \in \mathbb{R} \end{cases}$$

La quantità u valutata nel punto x varia nel tempo a seconda di quanto $u(x, \dots)$ differisce dalla sua media attorno a x



Contesto: Diffusione su un grafo

- Grafo topologico: insieme finito di «nodi» e di una relazione anti riflessiva simmetrica non transitiva su di essi
- Usiamo i due precedenti strumenti per creare diffusione di calore su un grafo
- Algebricamente parlando, con i grafi: $L = D - A$
- $(Lu)_i$ = somma delle differenze tra il nodo i e i suoi vicini
- L'equazione del calore diventa:

$$\begin{cases} \frac{\partial}{\partial t} \mathbf{u}(t) = -L\mathbf{u}(t) & t > 0 \\ \mathbf{u}(0) = \mathbf{u}^0 \end{cases}$$



Contesto: Diffusione su un grafo

- Abbiamo ottenuto un sistema di equazioni differenziali ordinarie (ODE) accoppiate dalla matrice Laplaciana
- Sfruttiamo la decomposizione spettrale $L = P\Lambda P^T$ per risolverla:

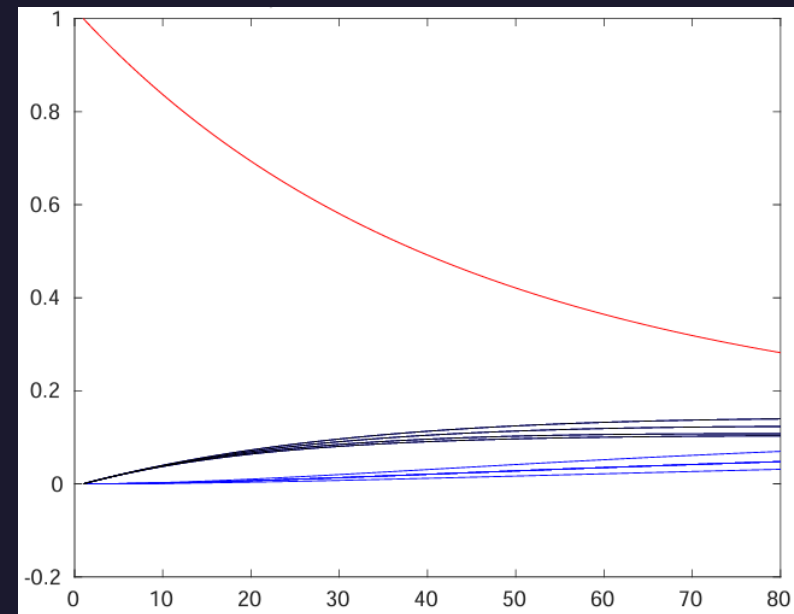
$$\begin{cases} \frac{d}{dt}v_j(t) = -\lambda v_j(t) & t > 0 \\ v_j(0) = P(:,j)^t \mathbf{u}^0 \end{cases} \Rightarrow v_j(t) = e^{-\lambda t} P(:,j)^t \mathbf{u}^0$$

- Risultato: somma pesata dei modi naturali di vibrazione del grafo che decadono nel tempo
$$\mathbf{u}(t) = P \cdot e^{-\Lambda t} \cdot P^T \cdot \mathbf{u}(0)$$
- Possiamo calcolare lo stato del sistema a qualsiasi tempo t usando l'algebra



Scopo del progetto: Ricostruzione di un grafo

- Un dato fenomeno evolve su un grafo per diffusione
- Non c'è conoscenza del grafo
- Ricostruzione possibile grazie a misurazioni di fenomeni diffusivi
- Idea: partendo a scaldare un nodo, i nodi vicini avrebbero veloce variazione di calore nel tempo, mentre quelli distanti avrebbero una lenta variazione
- Adatto per tempi sufficientemente brevi





Inizializzazione

buildsyntheticdata.m

- Parametri del problema: numero di nodi, numero di esperimenti, numero di istanti temporali, tempo finale di osservazione, ordine di grandezza del rumore, probabilità di creare un arco
- Generazione del grafo e del Laplaciano
- Generazione dei dati iniziali ($u(0)$)
- Simulazione della diffusione dei vari esperimenti fino al tempo finale
- Snapshots del grafo (le varie $u(\dots)$) per ogni istante temporale stabilito
- Salvataggio dei dati
- Vediamo il codice...

Tramite uso delle «cells» in MATLAB





Problema 1: dati diffusivi sovraabbondanti

Spiegazione del problema

Data la lista degli m nodi $V = \{v_1, \dots, v_m\}$, una serie finita di istanti temporali $0 = t_0 < t_1 < \dots < t_n = T$ ed la misurazione di $M \geq m$ processi diffusivi $U\{j\}(i, k) := u^{(k)}(v_i, t_j) + \epsilon_{i,j,k}$, con $i = 1, \dots, m, j = 1, \dots, n$ e $k = 1, 2, \dots, M$, e dove $\epsilon_{i,j,k}$ è un errore di misurazione, trovare la matrice di adiacenza A del grafo.

- Sistema lineare sovradeterminato: $Ax = b$ con $A \in M_{M \times m}$ con $M \geq m$
- Non esiste una soluzione esatta x che soddisfi $Ax = b$
- Obiettivo: cercare la x migliore possibile
- Strumento utilizzato: minimi quadrati con fattorizzazione SVD per minimizzare la norma del residuo $\|Ax - b\|_2$



solutore_1.m

- Risolve il problema I
- «A» = cells «U», «x» = «c», «b» = quale nodo prendiamo come partenza
- Calcolo delle traiettorie tramite calcolo del vettore «c» (con «pinv» per SVD) e delle derivate (velocità di diffusione)
- Costruzione della matrice di adiacenza (e di conseguenza del grafo) grazie a clustering (con «kmeans» in MATLAB)
- Vediamo il codice...
- Analizziamo quanto è stato efficiente il processo tramite «statistiche.m»...
- Visualizziamo graficamente i fenomeni grazie a «visualizza_dati.m»...





Problema 2: dati diffusivi lacunosi

Spiegazione del problema

Data la lista degli m nodi $V = \{v_1, \dots, v_m\}$, una serie finita di istanti temporali $0 = t_0 < t_1 < \dots < t_n = T$ ed la misurazione di $M < m$ processi diffusivi $U\{j\}(i, k) := u^{(k)}(v_i, t_j) + \epsilon_{i,j,k}$, con $i = 1, \dots, m, j = 1, \dots, n$ e $k = 1, 2, \dots, M$, e dove $\epsilon_{i,j,k}$ è un errore di misurazione, trovare la matrice di adiacenza A del grafo.

- Sistema lineare sottodeterminato: $Ax = b$ con $A \in M_{M \times m}$ con $M < m$
- Infinite soluzioni x che soddisfano esattamente $Ax = b$
- Obiettivo: cercare la x migliore possibile
- Strumento utilizzato: soluzione di minima norma con fattorizzazione SVD per cercare la x che ha la norma $\|x\|_2$ più piccola tra tutte le soluzioni valide



solutore_2.m

- Risolve il problema 2
- «A» = cells «U», «x» = «c», «b» = quale nodo prendiamo come partenza
- Calcolo delle traiettorie tramite calcolo del vettore «c» (con «pinv» per SVD) e delle derivate (velocità di diffusione)
- Problema: ci sono meno esperimenti rispetto ai nodi → soluzione: aumentiamo il numero di esperimenti artificialmente, utilizzando ogni istante temporale come un nuovo esperimento
- Costruzione della matrice di adiacenza (e di conseguenza del grafo) grazie a clustering (con «kmeans» in MATLAB)
- Vediamo il codice...
- Analizziamo quanto è stato efficiente il processo tramite «statistiche.m»...
- Visualizziamo graficamente i fenomeni grazie a «visualizza_dati.m»...



solutore_sliding.m

- Risolve sempre il problema 2, con la stessa logica di base, ma in maniera più robusta
- Migliore gestione del rumore:
 - Il precedente codice calcolava la derivata guardando solo due istanti consecutivi → derivata molto sensibile al rumore
 - Il nuovo codice prende in esame una finestra temporale, in cui esegue una regressione lineare per trovare la pendenza media → derivata molto più stabile
- Costruzione della cell « U »:
 - Il precedente codice concatenava gli snapshot dei singoli istanti ($U\{1\}, U\{2\}, U\{3\}, \dots$)
 - Il nuovo codice concatena i risultati delle finestre, estraendo media delle temperature e velocità di variazione calcolata con la regressione
- Le derivate vengono calcolate per ogni finestra subito, prima ancora di sapere i coefficienti di « c ».
- « c » viene calcolato basandosi sugli stati medi.
- Vediamo il codice...
- Analizziamo quanto è stato efficiente il processo tramite «statistiche.m»...
- Visualizziamo graficamente i fenomeni grazie a «visualizza_dati.m»...





Considerazioni finali

Considerazioni finali

- Come esplicitato su Moodle, era possibile contattare il prof. Piazzon per suggerimenti e dubbi
→ «solutore_sliding.m» perché ci interessava migliorare «solutore_2.m»
- Scelta di questo specifico progetto dovuta a:
 - Conoscenze pregresse sui grafi dal Corso del Primo Anno «Algebra e Matematica Discreta»
 - Progetto più interessante di tutti (a nostro parere)
 - Ambizioni future





FINE

Grazie dell'attenzione!